

Módulo 3: Segurança na Camada de Aplicação (Desenvolvimento)

1. Gestão de Sessão e Cookies: O que são Cookie SessionID e quais preocupações devo ter com eles, na posição de pessoa desenvolvedora de software?

O Cookie SessionID é um identificador único, gerado pelo servidor web logo após a autenticação bem-sucedida do usuário. Como o protocolo HTTP não mantém o estado entre requisições, esse token é enviado pelo navegador a cada nova requisição dentro do sistema, permitindo que o servidor reconheça o usuário sem exigir a senha repetidas vezes.

Na posição de pessoa desenvolvedora de software, o Cookie SessionID deve ser tratado com o mesmo rigor de uma senha credencial, exigindo muita atenção e cuidado, a fim de evitar ataques maliciosos contra o sistema. As principais preocupações técnicas incluem:

Sequestro de Sessão (Session Hijacking): Se um atacante obtiver a string do SessionID, ele pode injetá-la em seu próprio navegador e personificar a vítima imediatamente, burlando o processo de login.

Vulnerabilidade a Scripts (XSS): Falhas de Cross-Site Scripting na aplicação permitem que scripts maliciosos injetados no front-end leiam o objeto `document.cookie`, enviando o SessionID para servidores externos.

2. Qual o papel do HTTPS na proteção de cookies de sessão durante a transmissão na rede? O que acontece se a aplicação usar apenas HTTP?

O papel do HTTPS (HTTP Secure) é proteger todo o tráfego da aplicação através de um canal criptografado gerenciado pelos protocolos TLS/SSL. O TLS/SSL funciona como uma camada invisível de segurança que embaralha as informações antes de saírem do computador, garantindo que cabeçalhos, cookies, URLs e o corpo das páginas trafeguem de forma totalmente confidencial.

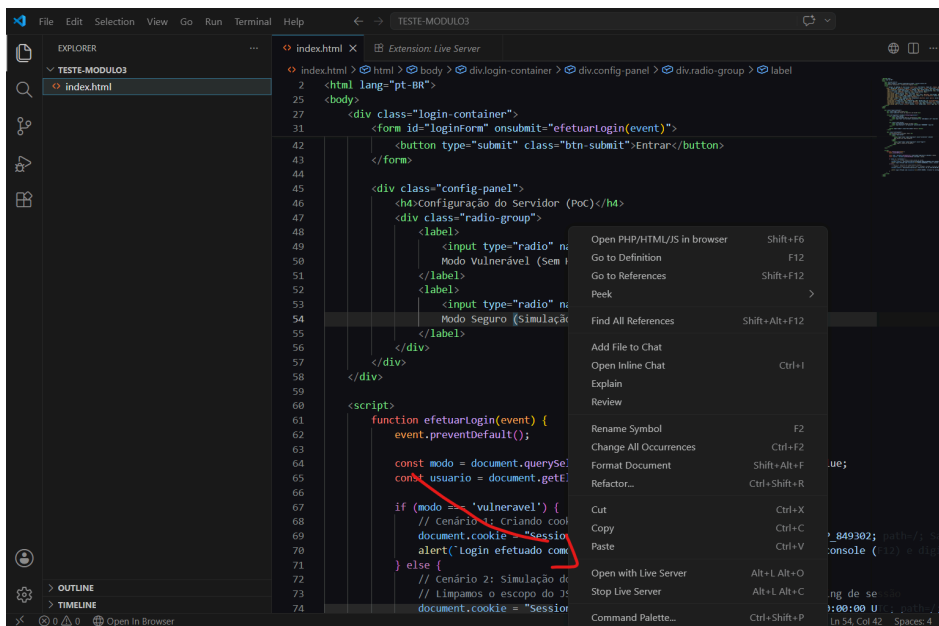
Caso a aplicação utilize apenas HTTP, todos os dados transitam pela rede em texto plano. Assim, um atacante posicionado na mesma rede local (LAN ou Wi-Fi pública) pode utilizar ferramentas de monitoramento de tráfego conhecidas como sniffers, por exemplo o Wireshark. Ao interceptar os pacotes de dados durante o tráfego deles, o invasor consegue ler o cabeçalho Cookie: SessionID=... sem qualquer dificuldade, capturando o acesso do usuário e podendo assim assumir o controle total dessa sessão.

3. Demonstração Prática (Prova de Conceito - PoC)

Ferramentas Usadas: Uma página de login criada em HTML5 e CSS3, usando o servidor local Live Server no Vs Code para rodar o site no endereço `http://127.0.0.1:5500`, e o Console do Navegador (F12) para testar os códigos.

Passo a Passo da Execução:

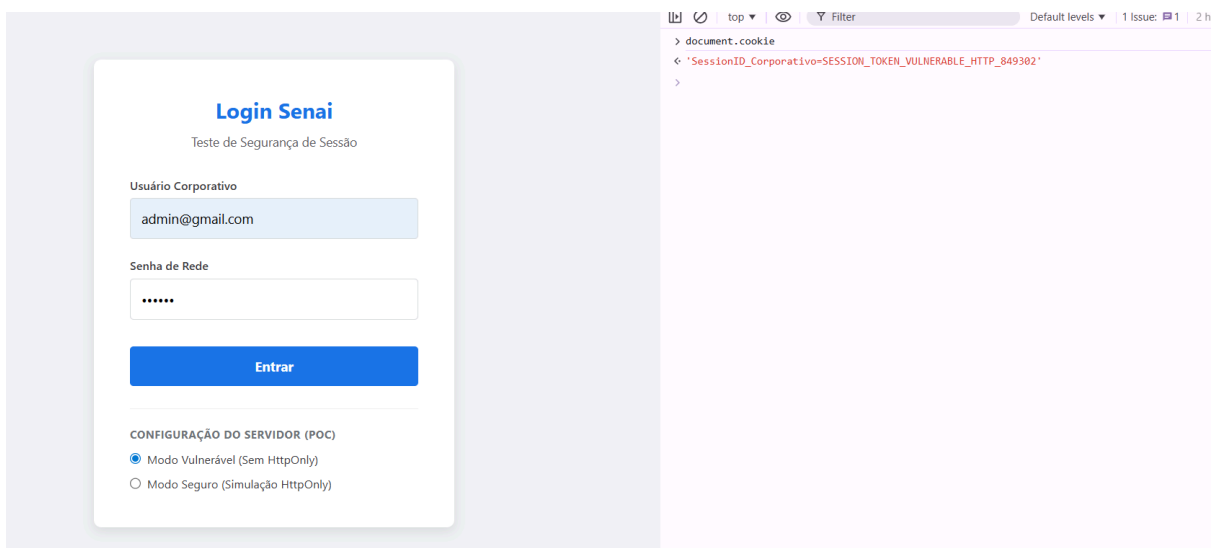
1 - O arquivo `index.html` foi aberto no VS Code e colocado para rodar através do Live Server.



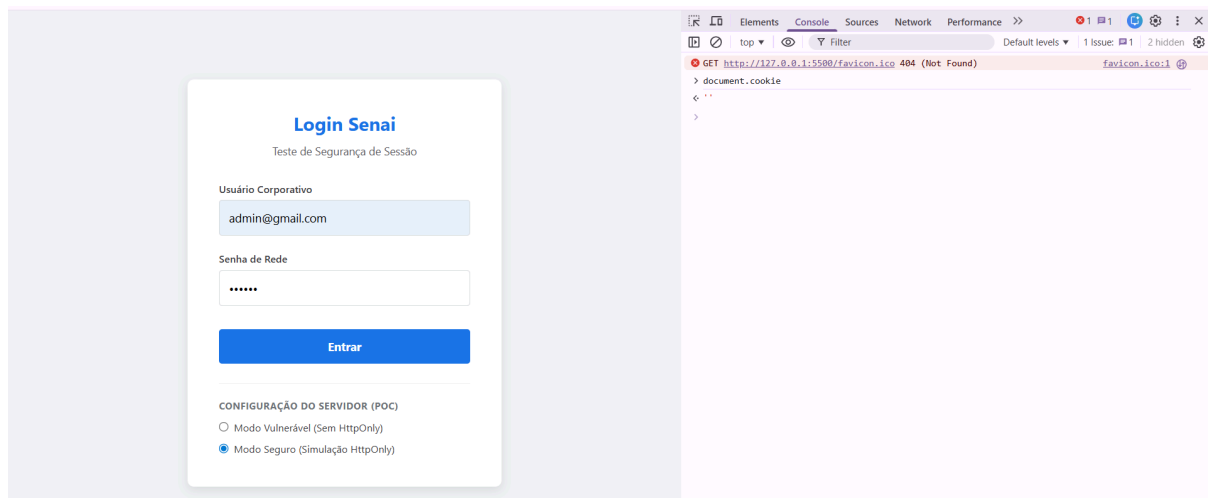
```
index.html
<html lang="pt-BR">
  <body>
    <div class="login-container">
      <form id="loginForm" onsubmit="efetuarLogin(event)">
        <button type="submit" class="btn-submit">Entrar</button>
      </form>
      <div class="config-panel">
        <h4>Configuração do Servidor (PoC)</h4>
        <div class="radio-group">
          <input type="radio" name="modo" checked="" value="vulneravel">
            Modo Vulnerável (Sem HttpOnly)
          <input type="radio" name="modo" value="seguro">
            Modo Seguro (Simulação HttpOnly)
        </div>
      </div>
    </div>
    <script>
      function efetuarLogin(event) {
        event.preventDefault();
        const modo = document.querySelector('input checked="" type="radio"');
        const usuario = document.getElementById('usuario').value;
        const senha = document.getElementById('senha').value;

        if (modo.value === 'vulneravel') {
          // Cenário 1: Criando cookie de sessão
          document.cookie = "SessionID_Corporativo=SESSION_TOKEN_VULNERABLE_HTTP_849302";
          alert("Login efetuado com sucesso!");
        } else {
          // Cenário 2: Simulação de cookie HttpOnly
          // Limpamos o escopo do JS
          document.cookie = "SessionID_Corporativo=SESSION_TOKEN_SECURE_HTTP_849302";
        }
      }
    </script>
  </body>
</html>
```

2 - Teste 1 (Modo Vulnerável): Marcamos a opção "Modo Vulnerável (Sem HttpOnly)", digitamos um usuário e senha e clicamos em entrar. Depois, abrimos o Console do navegador (F12), digitamos o comando `document.cookie` e apertamos Enter para ver se o JavaScript conseguia ler o cookie de sessão.



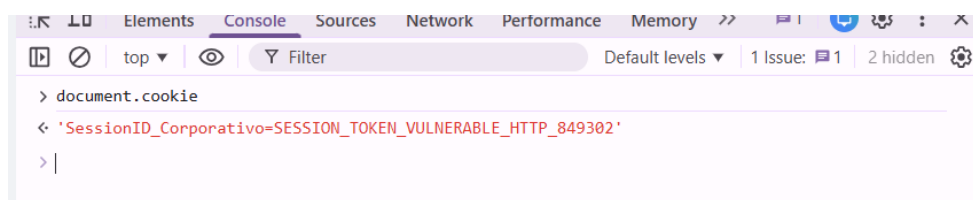
3 - Teste 2 (Modo Seguro): Marcamos a opção "Modo Seguro (Simulação HttpOnly)", fizemos o login, fomos ao Console (F12) e digitamos o mesmo comando `document.cookie` para checar se o acesso havia sido bloqueado.



Resultados:

Cenário 1: Aplicação Vulnerável (Falta da flag HttpOnly)

Sem a proteção ativada, qualquer invasor consegue ler o Cookie SessionID do usuário de login direto pelo console do navegador, permitindo assim um roubo de sessão. Dessa forma, ao usar o comando `document.cookie` é exibido o Cookie SessionID do usuário.



Cenário 2: Aplicação Protegida (Com HttpOnly ativo)

Ao ativar a simulação da proteção, o navegador esconde o cookie de sessão do console, inibindo um possível roubo do Cookie SessionID, protegendo dessa forma o tráfego do usuário pelo sistema. Assim, ao usar o comando `document.cookie` é exibida apenas uma string vazia.



Explicação Técnica:

O HTML e o ecossistema dos navegadores permitem que os dados sejam lidos por padrão através de scripts que rodam no próprio navegador (JavaScript). Quando uma aplicação não utiliza o **HttpOnly** — que é uma configuração de segurança aplicada ao cookie para proibir que códigos JavaScript tenham acesso a ele —, qualquer vulnerabilidade de injeção de script malicioso (XSS) dá ao atacante controle total sobre o SessionID através do comando `document.cookie`.

Ao ativarmos essa proteção, o navegador isola o cookie em uma área de memória interna e restrita. Isso cria uma barreira intransponível para qualquer script, garantindo que a sessão seja transmitida apenas de forma automática nas comunicações com o servidor, o que neutraliza completamente o impacto de um vazamento por XSS.